Name- Bhakare Purva Shivaji

Roll no- 08

Practical No- 02

MAZE SOLVING

```
from collections import deque
maze = [
   "S    #",
   "###  #",
   "#    #",
   "# ### ",
   "#    G"
]
start = (0, 0)
goal = (4, 6)
def bfs():
   q = deque([(start, [start])])
   visited = {start}
   while q:
      pos, path = q.popleft()
      if pos == goal:
         return path
      r, c = pos
      for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
         nr, nc = r + dr, c + dc
         if (0 <= nr < 5 and 0 <= nc < 7 and
            maze[nr][nc] != '#' and (nr,nc) not in visited):
            visited.add((nr,nc))
            q.append(((nr,nc), path + [(nr,nc)]))
print("Maze:")
for row in maze:
```

```
    print(row)


print("\nPath:", bfs())
```

OUTPUT-

```
Maze:
S     #
###   #
#     #
# ###
#     G
Path: [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (4, 5), (4, 6)]
```

ROUTE FINDING

```
from collections import deque
# Graph representing routes
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
def bfs(start, goal):
    queue = deque([[start]])
    visited = []
    while queue:
        path = queue.popleft()
        node = path[-1]
        if node == goal:
            return path
```

```
        if node not in visited:
            visited.append(node)

            for neighbour in graph[node]:
                new_path = list(path)
                new_path.append(neighbour)
                queue.append(new_path)
    return "No Route Found"
start = input("Enter Start Node: ")
goal = input("Enter Goal Node: ")
route = bfs(start, goal)
print("Route:", route)
```

OUTPUT-

```
Enter Start Node:  A
Enter Goal Node:  D
Route: ['A', 'B', 'D']
```